

230. 二叉搜索树中第 K 小的元素

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 树, 深度优先搜索, 二叉搜索树, 二叉树
- **Companies:** (Not Available)

给定一个二叉搜索树的根节点 `root` , 和一个整数 `k` , 请你设计一个算法查找其中第 `k` 个最小元素 (从 1 开始计数)。

示例 1:

输入 : `root = [3,1,4,null,2]` , `k = 1`

输出 : 1

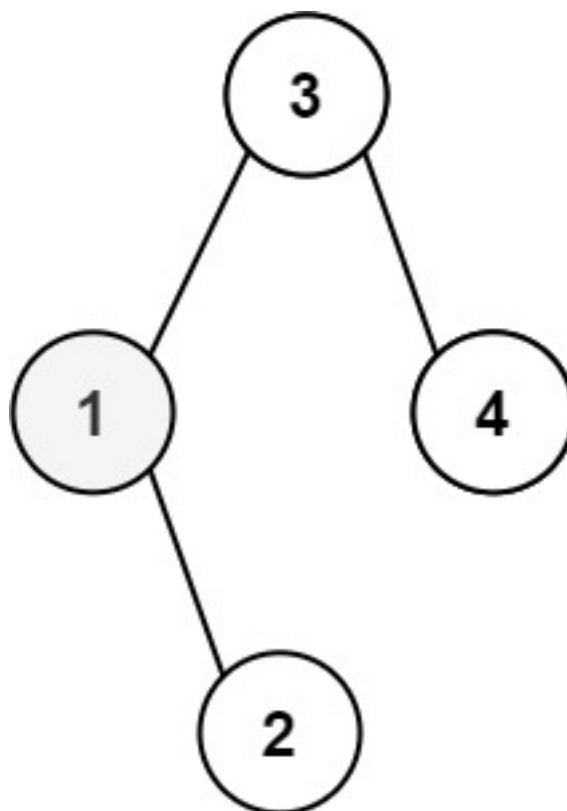


Figure 1: img

示例 2:

输入: root = [5,3,6,2,4,null,null,1], k = 3

输出: 3

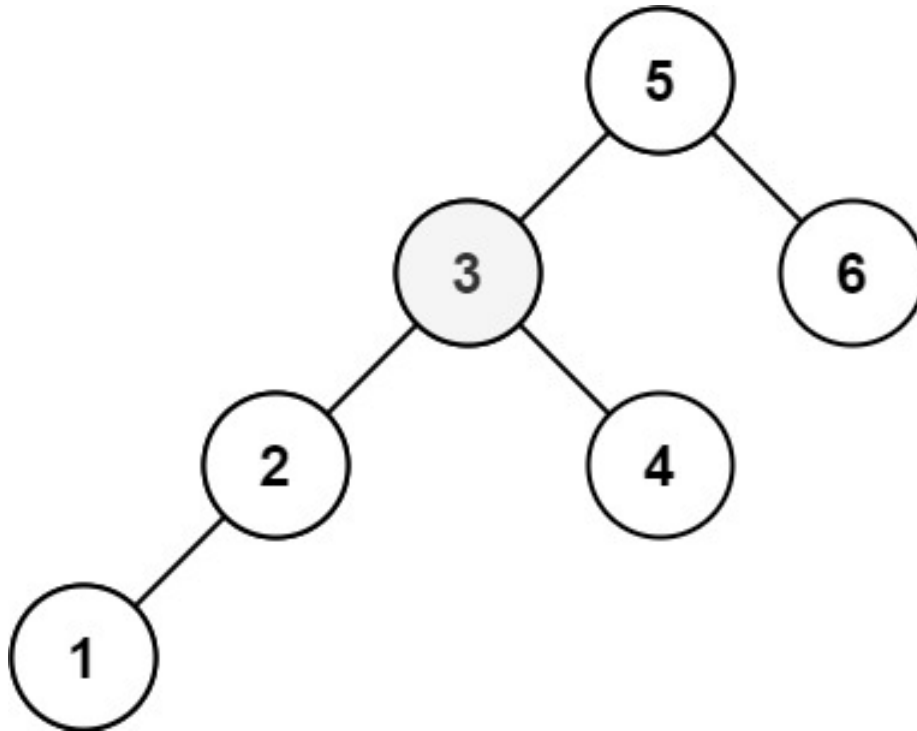


Figure 2: img

提示:

- 树中的节点数为 n 。
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

进阶: 如果二叉搜索树经常被修改 (插入/删除操作) 并且你需要频繁地查找第 k 小的值, 你将如何优化算法?

Discussion | Solution

解决方法

1. 问题理解

目标：在给定的二叉搜索树 (BST) 中找到第 k 小的元素。**BST 性质：**中序遍历结果是升序序列。**关键：**第 k 小的元素就是中序遍历序列中的第 k 个元素。

2. 思路分析

方法一：中序遍历 + 列表存储 (DFS - 递归) 最直接的方法是先获取 BST 的中序遍历结果 (一个升序列表)，然后直接返回列表中的第 $k-1$ 个元素 (因为列表索引从 0 开始)。

1. **初始化：**创建一个空列表 `inorder_list`。
2. **定义递归函数 `inorder(node)`：**
 - 执行标准的中序遍历：递归左子树 -> 将 `node.val` 加入 `inorder_list` -> 递归右子树。
3. **主函数调用：**
 - 调用 `inorder(root)` 填充 `inorder_list`。
 - 返回 `inorder_list[k - 1]`。

缺点：需要遍历整棵树并将所有节点值存储起来，空间复杂度为 $O(N)$ ，且即使第 k 小的元素在前面，也需要遍历完。

方法二：中序遍历 + 提前终止 (DFS - 递归) 我们可以在中序遍历的过程中进行计数，一旦找到第 k 个元素就立即停止遍历并返回结果。

1. **初始化：**
 - 使用一个计数器 `count`，初始为 0。
 - 使用一个变量 `result` 存储结果，初始为 `None` 或某个特殊值。
2. **定义递归函数 `inorder_find(node)`：**
 - **功能：**进行中序遍历，找到第 k 小的元素。
 - **基本情况 (Base Case)：**如果 `node` 为 `null`，返回。
 - **递归左子树：**`inorder_find(node.left)`。
 - **处理根节点：**
 - 计数器 `count` 加 1。
 - 检查 `count` 是否等于 k 。如果是，说明当前节点 `node` 就是第 k 小的元素，将其值赋给 `result`，然后**直接返回** (或设置一个标志位停止后续递归)。
 - **递归右子树：**如果 `result` 尚未找到 (`count < k`)，则递归调用 `inorder_find(node.right)`。

注意：`count` 和 `result` 需要是全局变量、实例变量或通过闭包/引用传递，以便在递归调用间共享状态。为了能提前终止，递归函数可以设计成找到结果后返回特殊信号，或者直接修改外部变量。

方法三：中序遍历 (迭代 - 栈) 使用迭代的方式进行中序遍历，同样可以在访问到第 k 个节点时停止。

1. **初始化：**`stack = []`, `curr = root`, `count = 0`。
2. **迭代循环：**`while curr or stack:`

- 一路向左, 将节点压栈: `while curr: stack.append(curr); curr = curr.left`。
 - 弹出节点访问: `node = stack.pop()`。
 - 计数: `count += 1`。
 - 检查: `if count == k: return node.val`。
 - 转向右子树: `curr = node.right`。
3. **返回结果**: 正常情况下循环会在找到第 k 个元素时返回。如果 k 无效 (例如大于节点数), 可能需要额外的错误处理, 但根据题目约束 $1 \leq k \leq n$, 总能找到。

这种方法空间复杂度为 $O(H)$, 并且可以在找到第 k 个元素后立即停止。

3. 代码实现 (Python)

```
# Definition for a binary tree node.
class TreeNode:
    #     def __init__(self, val=0, left=None, right=None):
    #         self.val = val
    #         self.left = left
    #         self.right = right

from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一: 中序遍历 + 列表 (递归)
    def kthSmallestList(self, root: Optional[TreeNode], k: int) -> int:
        inorder_list = []
        def inorder(node):
            if not node:
                return
            inorder(node.left)
            inorder_list.append(node.val)
            inorder(node.right)

        inorder(root)
        return inorder_list[k - 1]

    # 方法二: 中序遍历 + 提前终止 (递归)
    def kthSmallestRecursive(self, root: Optional[TreeNode], k: int) -> int:
        self.count = 0
        self.result = -1 # 或者 None
```

```

def inorder_find(node):
    if not node or self.result != -1: # 如果已找到结果, 提前终止
        return

    inorder_find(node.left)

    # 处理根节点
    self.count += 1
    if self.count == k:
        self.result = node.val
        return # 找到结果, 终止当前递归分支

    # 如果还没找到, 继续右子树
    if self.result == -1:
        inorder_find(node.right)

inorder_find(root)
return self.result

# 方法三: 中序遍历 (迭代)
def kthSmallestIterative(self, root: Optional[TreeNode], k: int) -> int:
    stack = []
    curr = root
    count = 0

    while curr or stack:
        # 一路向左
        while curr:
            stack.append(curr)
            curr = curr.left

        # 弹出节点访问
        node = stack.pop()

        # 计数并检查
        count += 1
        if count == k:
            return node.val

        # 转向右子树
        curr = node.right

    # 根据题目约束, k 总是有效的, 这里理论上不会执行到
    return -1 # 或者抛出异常

```

```
# 最终提交可以选择其中一种方法
def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
    # return self.kthSmallestList(root, k)
    # return self.kthSmallestRecursive(root, k)
    return self.kthSmallestIterative(root, k)
```

4. 复杂度分析

- **方法一（列表存储）：**
 - 时间复杂度： $O(N)$ ，需要遍历所有节点。
 - 空间复杂度： $O(N)$ ，需要存储所有节点值和递归栈（最坏 $O(N)$ ，平均 $O(\log N)$ ）。
- **方法二（递归提前终止）：**
 - 时间复杂度： $O(H + k)$ ，其中 H 是树的高度。在最坏情况下（ $k=N$ 或树呈链状），仍为 $O(N)$ 。平均情况下，如果 k 较小，会比 $O(N)$ 快。
 - 空间复杂度： $O(H)$ ，递归栈深度。
- **方法三（迭代）：**
 - 时间复杂度： $O(H + k)$ 。同上。
 - 空间复杂度： $O(H)$ ，栈深度。

结论：方法二和方法三在时间和空间上通常优于方法一，特别是当 k 远小于 N 时。迭代方法（方法三）避免了递归栈溢出的风险。

5. 如何在面试中讲解

1. **利用 BST 性质：**
 - “关键在于 BST 的中序遍历结果是升序的。所以第 k 小的元素就是中序遍历序列的第 k 个元素。”
2. **方法一：完整中序遍历 + 列表：**
 - “最简单的方法是先完整地进行中序遍历，将结果存入列表，然后返回列表第 $k-1$ 项。”
 - “缺点：需要 $O(N)$ 时间和 $O(N)$ 空间，效率不高，没有充分利用 BST 性质提前停止。”
3. **方法二/三：优化 - 中序遍历中计数：**
 - “我们可以优化，在中序遍历的过程中进行计数。当访问到第 k 个节点时，就找到了答案，可以立即停止。”
 - **递归实现（方法二）：**
 - “定义递归函数，使用全局变量或闭包来记录 `count` 和 `result`。”
 - “递归顺序：左 -> 根 -> 右。”
 - “访问根节点时 `count++`，如果 `count == k`，记录 `result` 并想办法终止后续递归（例如，检查 `result` 是否已赋值）。”
 - “复杂度：时间 $O(H+k)$ ，空间 $O(H)$ 。”
 - **迭代实现（方法三）：**
 - “使用栈模拟中序遍历。”
 - “迭代步骤：一路向左入栈 -> 弹出节点 `node` -> `count++` -> 如果 `count == k` 返回 `node.val` -> 处理右子树。”
 - “这是推荐的方法，时间 $O(H+k)$ ，空间 $O(H)$ ，且无递归栈溢出风险。”
4. **复杂度分析对比：**
 - “对比三种方法的时间和空间复杂度，强调方法二和三的优势。”

5. 进阶问题讨论:

- “如果需要频繁查找第 k 小, 并且树会修改, 怎么办?”
- “思路: 维护额外信息。可以在每个节点存储其左子树的大小 (节点数)。”
- “查找过程:
 - 设当前节点为 `node`, 其左子树大小为 `left_size`。
 - 如果 $k == \text{left_size} + 1$, 则当前节点 `node` 就是第 k 小。
 - 如果 $k \leq \text{left_size}$, 则第 k 小在左子树中, 递归查找 `kthSmallest(node.left, k)`。
 - 如果 $k > \text{left_size} + 1$, 则第 k 小在右子树中, 递归查找 `kthSmallest(node.right, k - (\text{left\_size} + 1))` (在右子树中查找第 $k - \text{left_size} - 1$ 小)。”
- “这种方法查找时间复杂度为 $O(H)$ 。但插入/删除时需要更新子树大小信息, 插入/删除复杂度变为 $O(H)$ 。这是一种空间换时间的优化, 适用于查找远多于修改的场景。”
- “也可以考虑使用平衡二叉搜索树 (如 AVL 树、红黑树) 并增强节点信息, 或者使用其他支持顺序统计的数据结构 (如 Order-Statistic Tree)。”